

Relatório Sockets

Os seis programas Python do slide *REDES3 – Sockets TCP (Parte 2)* foram implementados e testados em uma única máquina, e todos funcionaram como descrito no slide. Os servidores UDP e TCP devolveram as frases em maiúsculas, e o servidor concorrente atendeu dois clientes ao mesmo tempo.

Alteração em relação ao slide: Nos clientes, os nomes dos servidores: `serverName = 'hostname'` em `UDPClient.py` e `serverName = 'servername'` em `TCPClient.py` foram trocados para `127.0.0.1`. O restante do código é igual ao do slide.

Como executar. Cada par usa dois terminais: primeiro o servidor, depois o cliente.

`python UDP/UDPServer.py` (terminal 1)
`python UDP/UDPClient.py` (terminal 2)

`python TCP/TCPServer.py` (terminal 1)
`python TCP/TCPClient.py` (terminal 2)

`python TCPConc/TCPServerConc.py` (terminal 1)
`python TCPConc/TCPClientConc.py` (terminal 2)

1. UDP — UDPServer.py e UDPClient.py

No UDP: cada frase enviada pelo cliente voltou em maiúsculas, sem nenhuma conexão entre os dois processos.

UDPServer.py — código

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))
print('Servidor disponível')
while True:
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(),
                        clientAddress)
```

UDPServer.py — funcionamento

1. `socket(AF_INET, SOCK_DGRAM)` cria um socket IPv4 do tipo datagrama, ou seja, UDP.
2. `bind("", serverPort)` associa o socket à porta 12000.
3. No laço infinito, `recvfrom(2048)` bloqueia até chegar um datagrama de até 2048 bytes. Ele devolve os dados e o endereço de origem.
4. A mensagem é decodificada para texto, convertida com `upper()` e enviada de volta com `sendto` para `clientAddress`.

Não há conexão: o mesmo socket atende todos os clientes. O servidor só sabe para quem responder porque cada datagrama traz o endereço do remetente.

UDPClient.py — código

```
from socket import *
serverName = '127.0.0.1'
serverPort = 12000
clientSocket = socket(AF_INET,
                      SOCK_DGRAM)
message = input('Input lowercase sentence: ')
clientSocket.sendto(message.encode(),
                    (serverName, serverPort))
modifiedMessage, serverAddress = clientSocket.recvfrom(2048)
print(modifiedMessage.decode())
clientSocket.close()
```

UDPClient.py — funcionamento

1. `serverName` e `serverPort` identificam o processo servidor: IP `127.0.0.1` e porta 12000.
2. O sistema operacional escolhe uma porta para o cliente no primeiro envio.
3. `input` lê a frase do teclado e `sendto` envia os bytes e anexa o endereço de destino ao datagrama.
4. `recvfrom(2048)` espera a resposta. O programa imprime o texto recebido e fecha o socket.

Execução

Server: `python UDP/UDPServer.py`

The server is ready to receive

Cliente: `python UDP/UDPClient.py`.

```
from socket import *  
serverPort = 12000  
serverSocket = socket(AF_INET, SOCK_STREAM)  
serverSocket.bind(('', serverPort))  
serverSocket.listen(1)  
print('The server is ready to receive')  
while True:  
    connectionSocket, addr = serverSocket.accept()  
    sentence = connectionSocket.recv(1024).decode()  
    capitalizedSentence = sentence.upper()  
    connectionSocket.send(capitalizedSentence.  
                           encode())  
    connectionSocket.close()
```

TCPServer.py — funcionamento

1. `socket(AF_INET, SOCK_STREAM)` cria um socket TCP, usado só para receber pedidos de conexão.
2. `bind` associa o socket à porta 12000 e `listen(1)` o deixa à espera de conexões.
3. `accept()` bloqueia até um cliente se conectar. Ele devolve um socket novo, `connectionSocket`, só para aquele cliente, e o endereço dele.
4. `recv(1024)` lê até 1024 bytes enviados pelo cliente.
5. A frase é convertida com `upper()`, enviada com `send`, e `close()` fecha apenas a conexão com esse cliente.

O servidor atende um cliente por vez: só volta ao `accept()` depois de fechar a conexão atual.

TCPClient.py — código

```
from socket import *
serverName = '127.0.0.1'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName, serverPort))
sentence = input('Input lowercase sentence:')
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print('From Server:', modifiedSentence.decode())
clientSocket.close()
```

TCPClient.py — funcionamento

1. O socket é criado com `SOCK_STREAM`, ou seja, TCP.
2. `connect((serverName, serverPort))` estabelece a conexão com o servidor. O servidor precisa já estar rodando.
3. `input` lê a frase do teclado e `send` a envia, sem informar o endereço de destino, pois o socket já está conectado.
4. `recv(1024)` espera a resposta. O programa a imprime depois de From Server: e fecha o socket.

Execução

Server: `python TCP/TCPServer.py`

The server is ready to receive

```
import socket
import _thread

HOST = '127.0.0.1'    # Endereco IP do Servidor
PORT = 50000          # Porta que o Servidor está

# Função chamada quando uma nova thread for iniciada
def conectado(con, cliente):
    print('\nCliente conectado:', cliente)

    while True:
        # Recebendo as mensagens através da conexão
        msg = con.recv(1024)
        if not msg:
            break
```

```

        print('\nCliente..:', cliente)
        print('Mensagem..:', msg)

    print('\nFinalizando conexao do cliente', cliente)
    con.close()
    _thread.exit()

tcp = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
orig = (HOST, PORT)

# Colocando um endereço IP e uma porta no Socket
tcp.bind(orig)

# Colocando o Socket em modo passivo
tcp.listen(1)
print('\nServidor TCP concorrente iniciado no IP', HOST, 'na porta', PORT)

while True:
    # Aceitando uma nova conexão
    con, cliente = tcp.accept()
    print('\nNova thread iniciada para essa conexão')

    # Abrindo uma thread para a conexão
    _thread.start_new_thread(conectado, tuple([con, cliente]))

# Fechando a conexão com o Socket
tcp.close()

```

TCPServerConc.py — funcionamento

1. **HOST = '127.0.0.1'** e **PORT = 50000** fazem o servidor aceitar só conexões da própria máquina, na porta 50000.
2. A thread principal cria o socket TCP, faz **bind** e **listen** e entra no laço de **accept()**.
3. A cada conexão aceita, **_thread.start_new_thread** cria uma thread que executa **conectado** para aquele cliente. A thread principal volta na mesma hora ao **accept()**.
4. Dentro da thread, **con.recv(1024)** recebe as mensagens, que são impressas como bytes, no formato **b'...'**.
5. Quando o cliente fecha a conexão, **recv** devolve bytes vazios. A thread fecha o socket e termina com **_thread.exit()**.

Este servidor só imprime o que recebe e não envia resposta ao cliente. O `tcp.close()` da última linha nunca é executado, porque o `while True` não termina.

TCPClientConc.py — código

```
import socket

HOST = '127.0.0.1' # Endereco IP do Servidor
PORT = 50000       # Porta que o Servidor está

# Criando a conexão
tcp = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
destino = (HOST, PORT)
tcp.connect(destino)

print('\nDigite suas mensagens')
print('Para sair use CTRL+X\n')

# Enviando as mensagens para o Servidor TCP através da conexão
try:
    # Recebendo a mensagem do usuário final pelo teclado
    mensagem = input()
    while '\x18' not in mensagem:
        tcp.send(str(mensagem).encode())
        mensagem = input()
# Fechando o Socket
tcp.close()
```

TCPClientConc.py — funcionamento

1. O cliente cria um socket TCP e conecta em `127.0.0.1`, porta 50000.
2. Ele imprime as instruções e lê uma linha do teclado com `input()`.
3. Enquanto a linha não tiver o caractere de CTRL+X (`\x18`), ele a envia com `tcp.send` e lê a próxima, sempre pela mesma conexão.
4. CTRL+X seguido de ENTER encerra o laço. `tcp.close()` fecha a conexão.

O cliente só envia: ele não chama `recv` em nenhum momento.

Execução

Server: `python TCPConc/TCPServerConc.py`

Servidor TCP concorrente iniciado no IP 127.0.0.1 na porta 50000

Nova thread iniciada para essa conexão

Cliente conectado: ('127.0.0.1', 53507)

Nova thread iniciada para essa conexão

Cliente conectado: ('127.0.0.1', 53508)

Cliente...: ('127.0.0.1', 53507)

Mensagem.: b'Ola, sou o cliente 1'

Cliente...: ('127.0.0.1', 53508)

Mensagem.: b'Ola, sou o cliente 2'

Cliente...: ('127.0.0.1', 53507)

Mensagem.: b'segunda mensagem do cliente 1'

Cliente...: ('127.0.0.1', 53508)

Mensagem.: b'segunda mensagem do cliente 2'

Finalizando conexao do cliente ('127.0.0.1', 53507)

Cliente...: ('127.0.0.1', 53508)

Mensagem.: b'cliente 2 ainda conectado depois que o 1 saiu'

Finalizando conexao do cliente ('127.0.0.1', 53508)

Cliente 1: `python TCPConc/TCPClientConc.py`.

Digite suas mensagens

Para sair use CTRL+X e ENTER

Ola sou o cliente 1

segunda mensagem do cliente 1

^X

Cliente 2: `python TCPConc/TCPClientConc.py`.

Digite suas mensagens

Para sair use CTRL+X

Ola sou o cliente 2

segunda mensagem do cliente 2

cliente 2 ainda conectado depois que o 1 saiu

^X

Os dois clientes ficaram conectados ao mesmo tempo, em portas diferentes, e o servidor criou uma thread para cada um. As mensagens chegaram intercaladas, e o cliente 2 continuou sendo atendido depois que o cliente 1 saiu.